Università degli Studi di Verona

CORSO DI LAUREA IN INFORMATICA

Fondamenti dell'Informatica

Federico Brutti
federico.brutti@studenti.univr.it

Inserire citazione inerente alla materia

Contents

1 Introduzione **3**
1.1 Storia dell'informatica 3
1.2 Contestualizzazione 4
1.3 Problema dell'Informatica 5
1.4 Concetti di base richiesti 6

2 Automi e linguaggi formali **7**
2.1 Linguaggi e grammatiche 7
2.2 Automi a stati finiti e linguaggi regolari 7
2.3 Linguaggi liberi da contesto, forme normali e automi a pila 7
2.4 Classificazione di Chomsky 7

3 Teoria della calcolabilità **8**

Chapter 1

Introduzione

1.1 Storia dell'informatica

L'**informatica**, abbreviazione di informazione automatica è una scienza nata negli anni '60 dalla fusione di vari aspetti scientifici e ingegneristici. In particolare ha un'origine **matematica**, dove si è definito che cosa sia "calcolare", e una **tecnologica**, determinante gli strumenti utilizzati per il calcolo. Quest'ultimo, nel senso moderno, nasce nell'epoca dell'Illuminismo ed è dovuto, in ambito del Determinismo, alla necessità di approssimare logaritmi e funzioni trigonometriche mediante polinomi, ottenendo un risultato univoco a partire da date condizioni iniziali.

Formalizzate queste tecniche nel '700, si vide successivamente la necessità di **meccanizzare** il calcolo. Il primo strumento funzionante fu la **macchina analitica**, creata da Charles Babbage e Ada Lovelace nel 19° secolo; proponeva un concetto rivoluzionario: la possibilità di programmazione per poter ritornare risultati in base ai parametri forniti. Poteva essere usata per sviluppare qualsiasi funzione di complessità e generalità arbitrarie, ripetendo operazioni sugli stessi elementi.

Le origini matematiche dell'informatica ebbero invece luogo con la necessità di verificare se la matematica fosse formalizzabile come insieme finito non contraddittorio di assiomi. Ciò non è possibile, perché ci si scontra inevitabilmente con le contraddizioni e gli assurdi. L'effettivo limite di ciò che è calcolabile e non fu definito con il **teorema di incompletezza** di Gödel; la restrizione del range di studio fu fondamentale per avere un'idea più chiara del dominio operativo. Andando avanti con gli anni, Alan Turing ideò l'omonima macchina, la cui idea principale era la possibilità interpretare il **calcolo** come sequenza anche infinita di sostituzioni di simboli: un algoritmo. Questo era naturalmente un programma finito dove ogni passo dipende da altrettante informazioni finite. Separa inoltre il controllo dalla memoria, ponendo le basi per quello che sarebbero diventati i calcolatori moderni. Oltre a ciò, insieme a Church definì la **Tesi di Church-Turing**, la quale afferma che se un problema è intuitivamente calcolabile, allora esisterà una

macchina di Turing (MDT) o un dispositivo equivalente[1] in grado di risolverlo, ovvero calcolarlo. L'ambito di studio dell'informatica è interamente incluso nell'insieme delle funzioni calcolabili, ovvero i problemi, le cui soluzioni sono fornite dagli algoritmi.

1.2 Contestualizzazione

Studiare la teoria della ricorsione e della calcolabilità è utile per fornire consapevolezza in merito ai limiti logici assoluti; insegna in primo luogo a riconoscere problemi impossibili, fornisce un criterio per capire quando fermarsi e permette di partire da assunzioni più chiare durante lo sviluppo software.

Tutte queste competenze sono in ogni caso usate durante il **calcolo**, formalmente definito come la risoluzione di un problema tramite un algoritmo, indipendentemente da quanto tempo o spazio è richiesto per la computazione. Come già menzionato, tuttavia, non sarà sempre possibile trovare una soluzione per ogni problema proposto, anche se formalmente definibile; per questa ragione, in termini pratici, non esistono programmi che verifichino la correttezza assoluta o la sicurezza di un dato software. Ciò che più si avvicina allo scenario di controllo ideale è l'**analisi del software**, ovvero la verifica del comportamento del programma in base a ciò che ci si aspetta. In particolar modo bisogna garantire:

- **Affidabilità**: Il software non deve andare in crash improvvisamente.
- **Sicurezza**: Il software, interfacciandosi con utenti esterni, non deve essere violato o manipolato.
- **Protezione**: Il software, se in controllo di sistemi critici, non deve causare incidenti.

Idealmente, dato un qualunque software con una data proprietà di interesse, esisterebbe un programma di analisi atto al calcolo automatico e preciso del soddisfacimento della stessa in tempo finito: il debugger ideale. Tuttavia, non è realistico che esista, poiché non abbiamo modo di verificare a priori se un programma termini o meno; in altre parole non può esistere:

$$\text{Per ogni programma } p \in \mathbb{L},\ \text{halt}(p) = \text{true} \iff p \text{ termina}$$

Ciò vale anche per un eventuale programma antivirus ideale, che segnalerebbe correttamente ogni malware nel caso in cui i relativi programmi in questione eseguissero compiti malevoli, come rubare o cifrare i dati. Questo perché non è possibile creare un algoritmo universale per un'analisi infallibile; tenendo a mente la definizione appena discussa, supponiamo un malware M che esegue un secondo programma P su un dato input x: se $P(x)$

[1] Strumenti come lambda-calcolo, funzioni ricorsive o linguaggi di programmazione turing completi.

termina, un file viene rimosso, altrimenti non arriva mai alla cancellazione. Non potendo sapere se un programma termini a priori, non sarà altrettanto possibile segnalarlo come malware, mostrando il limite logico.

Ancora più difficile sarebbe dimostrare che un software sta agendo in modo malevolo quando il discriminante è una proprietà semantica non banale, come la cifratura dei dati, nel caso dei ransomware. Da ciò, il **Teorema di Rice**: non esiste un algoritmo tale che per ogni programma sia capace di decidere automaticamente una proprietà semantica non banale, come per esempio la terminazione, fornire un risultato specifico, o determinare se due computazioni siano equivalenti.

Ciò non significa che tutto sia precluso; bisogna accettare i limiti imposti dall'insieme dei problemi calcolabili per lavorare esclusivamente su questi ultimi. In questi termini, non sarà possibile avere analisi automatiche corrette e complete, come anche ottenere risposte sempre precise, data dalla possibilità del verificarsi di falsi negativi e positivi. In ogni caso, esistono più iter di lavoro per l'analisi del software:

- **Model checking**: Controllo esaustivo di un modello del programma da analizzare.
- **Analisi statica**: Controllo automatico ed esaustivo sul programma senza che venga eseguito.
- **Testing**: Verifica di un sottoinsieme delle esecuzioni del sistema.
- **Antivirus firma e sandbox**: Sistema difensivo probabilistico, vulnerabile a stealthing.

1.3 Problema dell'Informatica

Prima di definire formalmente il problema che l'Informatica si pone di risolvere, è necessario chiarire alcuni concetti preliminari: sappiamo in primo luogo che il dominio di lavoro è un sottoinsieme dell'insieme dei numeri naturali $\mathbb{N}$ contenente le funzioni calcolabili, le quali sono definite come un insieme di coppie appartenenti alle parti di $\mathbb{N}$, ovvero:

$$f \in \mathcal{P}(\mathbb{N} \times \mathbb{N})$$

Per conoscere quante sono le funzioni del suddetto dominio, quindi la **cardinalità** dell'insieme $\mathcal{P}(\mathbb{N} \times \mathbb{N})$, notiamo che il numero di elementi all'interno di $\mathbb{N} \times \mathbb{N}$ ed $\mathbb{N}$ è equivalente e come conseguenza diretta sono entrambi numerabili e ordinabili. In questi termini esiste quindi una biezione univoca tra i due insiemi, ovvero che a ogni input corrisponde un output. La cardinalità dell'insieme delle parti $\mathcal{P}(\mathbb{N} \times \mathbb{N})$ sarà a sua volta equivalente a quella delle funzioni $\mathbb{N} \times \mathbb{N}$, dimostrando che esistono tante funzione quanti sottoinsiemi di $\mathbb{N}$.

Si ritiene ora necessario capire quanti algoritmi risolutivi esistano, definiti in termini logici come sequenze di simboli su un alfabeto Σ, il quale deve essere finito e non vuoto; esisteranno quindi tanti programmi quante sono le stringhe componibili con il suddetto alfabeto, il cui insieme è indicato con Σ^*. Per esempio:

$$\Sigma = \{a, b\} \implies \Sigma^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, ...\}$$

Dove ϵ rappresenta la stringa vuota. Si tratta di un insieme ordinabile privo di un limite superiore e in quanto tale può essere messo in relazione con quello dei naturali; ne concludiamo che:

$$|\Sigma^*| = |\mathbb{N}|$$

Dunque la cardinalità dell'insieme degli algoritmi è equivalente a quella dell'insieme dei naturali. Inoltre, per il **teorema di Cantor**, si può dimostrare come l'insieme delle parti avrà sempre cardinalità maggiore rispetto a quello intero; per esempio:

$$S = \{a, b\} \implies \mathcal{P}(S) = \{\{a\}, \{b\}, \emptyset, \{a, b\}\} \implies |S| = 2; |\mathcal{P}(S)| = 4$$

1.4 Concetti di base richiesti

Chapter 2

Automi e linguaggi formali

2.1 Linguaggi e grammatiche

2.2 Automi a stati finiti e linguaggi regolari

2.3 Linguaggi liberi da contesto, forme normali e automi a pila

2.4 Classificazione di Chomsky

Chapter 3

Teoria della calcolabilità